

CodeGen Project Plan v5 — Dual-Track: Text-to-SQL + Text-to-NoSQL

Grounded in: SchemaRAG (SIGMOD 2026) · TEND (Text-to-NoSQL) · Project Proposal · TextToSQL_Plan_v4

SECTION 1 — PROJECT OVERVIEW

What We Are Building

A unified natural-language-to-query system with two operational tracks:

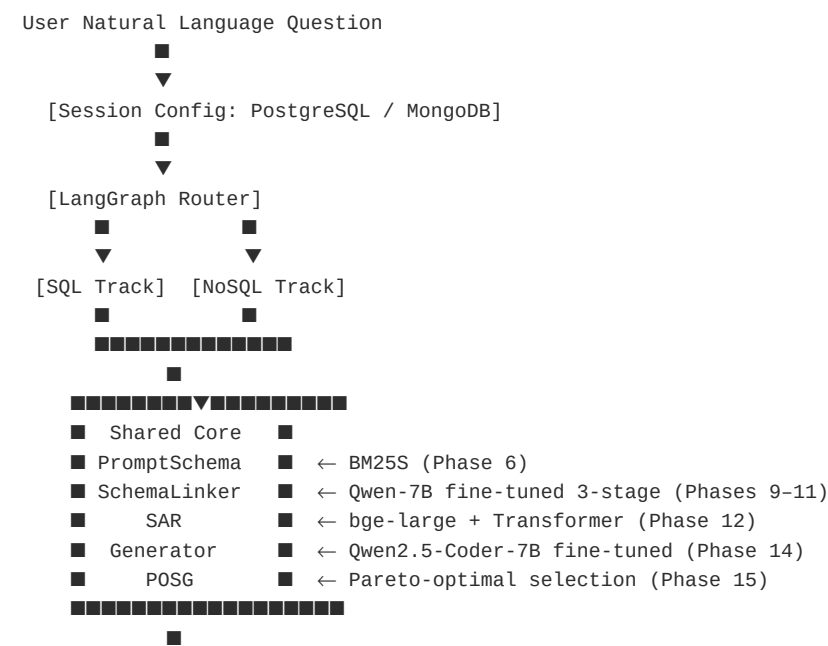
Track	Input	Output	Target DB	Primary Paper
Text-to-SQL	Natural language	PostgreSQL/SQLite SQL	PostgreSQL	SchemaRAG (SIGMOD 2026)
Text-to-NoSQL	Natural language	MongoDB MQL	MongoDB	TEND / SMART

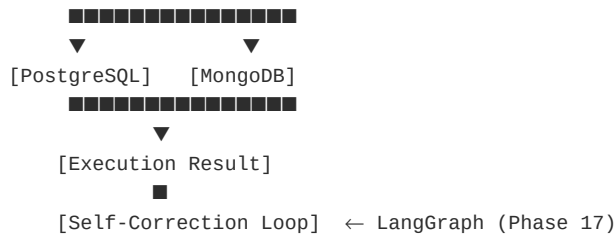
Both tracks share a common backbone architecture (SchemaLinker → SAR → Generator → POSG) and are unified under a LangGraph orchestration layer.

Routing Strategy (Option A + Twist)

- **Session-based**: User selects PostgreSQL or MongoDB at session start
- **Fallback detection**: If input looks like an existing SQL query (regex: SELECT/INSERT/UPDATE), route to SQL-to-NoSQL migration utility (CP4)
- **No conversational routing overhead**: cleaner, easier to test, sufficient for capstone

Architecture Diagram





NOTE on "Shared" vs "Split"

- Architecture is shared but models are trained SEPARATELY per track:
- SchemaLinker SQL checkpoint ≠ SchemaLinker NoSQL checkpoint
 - SAR SQL index ≠ SAR NoSQL index
 - Generator SQL fine-tune ≠ Generator NoSQL fine-tune

This is deliberate — the TEND paper shows direct generation outperforms cascade by 20+ points precisely because NoSQL queries have fundamentally different structure.

SECTION 2 — HARDWARE STRATEGY

Work	Where	Why
Data prep, FK graphs, BM25S, MongoDB conversion	Mac M1	No GPU needed; Python 3.10 + conda
CoT generation (API calls to GPT-4o / DeepSeek)	Mac M1	Network I/O, no GPU
SchemaLinker Stage 1 SFT	Colab T4 (free)	16GB fits Qwen-7B with LoRA r=16
SchemaLinker Stage 2 MTL	Colab T4 (free)	Same
SchemaLinker Stage 3 GRPO	Colab A100 (Pro)	G=8 samples × Qwen-7B ≈ 28GB min
SAR training	Colab T4 (free)	bge-large encoder + small Transformer
Qwen2.5-Coder-7B SFT (Generator)	Colab A100 (Pro)	7B + LoRA needs 24GB+ for batch training
Inference / pipeline testing	Mac M1 (4-bit GGUF) or Colab T4	
LangGraph assembly, POSG, demo	Mac M1	No GPU

Checkpoint strategy: All Colab-trained models saved to Google Drive at /content/drive/MyDrive/codegen/checkpoints/.

Mac ↔ Colab workflow: Write training scripts on Mac → git push → git pull on Colab → train → save to Drive.

SECTION 3 — COMPLETED PHASES

Phase	Description	Status
1	Project planning and literature review	■ Done

Phase	Description	Status
2	Architecture design decisions (Option A with twist)	■ Done
3A	Create text2sql conda env, Python 3.10	■ Done
3B	Install PyTorch for M1 MPS (no CUDA URL)	■ Done
3C	Clone SchemaRAG repo, install deps (skip av package)	■ Done

Phase 3D–3E still pending — complete these before Phase 4.

SECTION 4 — PHASE PLAN (FROM PHASE 3D)

Track annotations:

- = Shared foundation (same work benefits both tracks)
- = SQL-only work
- = NoSQL-only work
- = Same work done separately for each track

PHASE 3D — Device Helper [■, Mac, Immediate]

Create src/device.py:

```
import torch

def get_device():
    if torch.cuda.is_available():
        return "cuda"
    if hasattr(torch.backends, "mps") and torch.backends.mps.is_available():
        return "mps"
    return "cpu"

DEVICE = get_device()
print(f"Running on: {DEVICE}")
```

Verify: python src/device.py prints Running on: mps

PHASE 3E — Project Structure + Config [■, Mac, Immediate]

Create the full directory tree:

```
Codegen/
■■■ data/
■   ■■■ spider/           # Spider dataset (train, dev, tables, databases)
■   ■■■ fk_graphs/        # FK graph JSONs for 166 databases
■   ■■■ mongodb/          # MongoDB collection configs + sample docs
■   ■■■ prompt_schema/    # BM25S PromptSchema outputs (SQL + NoSQL)
■   ■■■ cot_data/         # CoT training data for SchemaLinker
■   ■■■ rag_corpus/       # Q-SQL and Q-MQL retrieval corpora
■■■ src/
■   ■■■ device.py
```

- ■■■■ fk_graph.py
- ■■■■ prompt_schema.py
- ■■■■ mongodb_converter.py
- ■■■■ schema_linker/
 - ■ ■■■■ train_stage1.py
 - ■ ■■■■ train_stage2.py
 - ■ ■■■■ train_stage3_grpo.py
 - ■ ■■■■ infer.py
- ■■■■ sar/
 - ■ ■■■■ sar_model.py
 - ■ ■■■■ train.py
 - ■ ■■■■ infer.py
- ■■■■ generator/
 - ■ ■■■■ train.py
 - ■ ■■■■ infer.py
- ■■■■ posg/
 - ■ ■■■■ posg_sql.py
 - ■ ■■■■ posg_nosql.py
- ■■■■ router/
 - ■■■■ langgraph_router.py
- scripts/ # Standalone Colab training scripts
- models/ # Trained checkpoints (local lightweight ones)
- indexes/ # ChromaDB vector stores
 - ■■■■ chroma_sql/
 - ■■■■ chroma_nosql/
- configs/
 - ■■■■ config.yaml
- evaluation/ # Eval scripts + results
- external/
- ■■■■ SchemaRAG/
- interface/ # Streamlit + FastAPI demo

Create configs/config.yaml:

```
dataset:
  name: spider
  train_path: data/spider/train_spider.json
  dev_path: data/spider/dev.json
  db_path: data/spider/database/
  tables_path: data/spider/tables.json

fk_graph:
  cache_path: data/fk_graphs/
  max_hops: 2
  fuzzy_threshold: 75
  top_k_seeds: 3

prompt_schema:
  sql_cache: data/prompt_schema/sql/
  nosql_cache: data/prompt_schema/nosql/
  top_k_values: 3

schema_linker:
  base_model: Qwen/Qwen2.5-7B
  sql_checkpoint: models/schema_linker_sql/
  nosql_checkpoint: models/schema_linker_nosql/
  self_consistency_k: 5 # k=5 for eval, k=1 for demo

sar:
  encoder_model: BAAI/bge-large-en-v1.5
  sql_checkpoint: models/sar_sql/
  nosql_checkpoint: models/sar_nosql/
  top_k: 3
  transformer_layers_sql: 3 # from SchemaRAG Fig 14: deeper for Spider
```

```
transformer_heads_sql: 8
transformer_layers_nosql: 2 # leaner for smaller NoSQL dataset
transformer_heads_nosql: 2
lr: 1.0e-4
contrastive_temperature: 0.05
```

```
generator:
  base_model: Qwen/Qwen2.5-Coder-7B-Instruct
  sql_checkpoint: models/generator_sql/
  nosql_checkpoint: models/generator_nosql/
  n_candidates: 5
  temperature: 0.8
  top_p: 0.95
  max_new_tokens: 2048
```

```
indexes:
  sql_chroma: indexes/chroma_sql/
  nosql_chroma: indexes/chroma_nosql/
```

```
mongodb:
  host: localhost
  port: 27017
  schema_cache: data/mongodb/
```

```
evaluation:
  results_dir: evaluation/results/
```

Create .env:

```
OPENAI_API_KEY=your_key_here
HF_TOKEN=your_hf_token_here
DEEPSEEK_API_KEY=your_key_here
```

Create .gitignore (exclude: data/, models/, indexes/, .env, __pycache__, *.ckpt).

Tasks:

1. Create all directories
2. Create config.yaml
3. Create .env and .gitignore
4. git add -A && git commit -m "Phase 3E: project structure" && git push

Verify: ls -la shows all directories; git log --oneline shows commit

PHASE 4 — Spider Dataset Acquisition [■, Mac]

Why this matters: Spider is the foundation for BOTH tracks. The 166 SQLite databases are used directly for Text-to-SQL AND will be converted to MongoDB format for Text-to-NoSQL.

What to download:

- train_spider.json — 7,000 (question, SQL, db_name) training examples
- dev.json — 1,034 development examples (our evaluation benchmark)
- tables.json — schema metadata for all 166 databases
- database/ folder — 166 SQLite database files

Download commands:

```
# Spider is available via Yale's drive or Hugging Face
# Option 1: Hugging Face dataset
pip install datasets
python -c "
from datasets import load_dataset
```

```

ds = load_dataset('spider')
"

# Option 2: Direct download from GitHub release
cd ~/Desktop/AIML\ Project.nosync/Codegen/data/spider
wget https://drive.google.com/uc?id=1iRDVHLr4mX2wQKSgA9J8Pire73Jahh0f
unzip spider.zip

```

Write scripts/validate_spider.py:

```

import json, sqlite3, os

# Check train + dev counts
train = json.load(open('data/spider/train_spider.json'))
dev = json.load(open('data/spider/dev.json'))
print(f"Train: {len(train)} (expect 7000)")
print(f"Dev: {len(dev)} (expect 1034)")

# Check databases
db_path = 'data/spider/database'
dbs = [d for d in os.listdir(db_path) if os.path.isdir(f"{db_path}/{d}")]
print(f"Databases: {len(dbs)} (expect 166)")

# Spot-check 5 databases
for db_name in dbs[:5]:
    db_file = f"{db_path}/{db_name}/{db_name}.sqlite"
    conn = sqlite3.connect(db_file)
    tables = conn.execute("SELECT name FROM sqlite_master WHERE type='table'").fetchall()
    print(f"  {db_name}: {len(tables)} tables")
    conn.close()

```

CP1 Deliverable: Spider dataset validated and available locally.

Verify: Script prints Train: 7000, Dev: 1034, Databases: 166

PHASE 5A — FK Graph Builder [■, Mac]

Why this is foundational for BOTH tracks:

- **SQL track:** FK relationships tell the SchemaLinker which tables are likely to JOIN — critical for predicting multi-table queries
- **NoSQL track:** The TEND paper (Algorithm 1) uses FK analysis to decide MongoDB document structure — whether to embed or reference related data

Implementation (src/fk_graph.py):

```

import sqlite3, json, networkx as nx
from rapidfuzz import fuzz

class FKGraphBuilder:
    def build(self, db_path: str, db_name: str) -> dict:
        conn = sqlite3.connect(f"{db_path}/{db_name}/{db_name}.sqlite")

        # Extract tables + columns
        tables = self._get_tables(conn)

        # Build NetworkX DiGraph
        G = nx.DiGraph()
        for table in tables:
            G.add_node(table['name'], columns=table['columns'], pk=table['pk'])

        # Add FK edges: (child_table, parent_table, {child_col, parent_col})
        for table in tables:

```

```

    for fk in self._get_foreign_keys(conn, table['name']):
        G.add_edge(table['name'], fk['parent_table'],
                    child_col=fk['child_col'],
                    parent_col=fk['parent_col'])

    # Compute centrality (which tables are most referenced)
    centrality = nx.in_degree_centrality(G)

    # Serialize
    return {
        'nodes': [{'name': n, **G.nodes[n]} for n in G.nodes],
        'edges': [{'from': u, 'to': v, **G.edges[u,v]} for u,v in G.edges],
        'centrality': centrality
    }

```

Build for all 166 databases → cache to data/fk_graphs/{db_name}.json

Key parameters (from config.yaml):

- max_hops: 2 — how many FK hops to include in schema linking context
- fuzzy_threshold: 75 — rapidfuzz threshold for matching ambiguous column names across tables

Verify: FK graph for concert_singer shows edges: singer_in_concert→singer, singer_in_concert→concert, concert→stadium. Centrality shows singer_in_concert as bridge table.

PHASE 5B — MongoDB Collection Generator [■, Mac]

Why: The TEND paper converted Spider's 166 SQLite databases into MongoDB format to create the Text-to-NoSQL training dataset. We must replicate this to have a NoSQL training corpus. There is no pre-built public version of this dataset.

Install MongoDB on Mac first:

```

brew tap mongodb/brew
brew install mongodb-community
brew services start mongodb-community
pip install pymongo --break-system-packages

```

Conversion Algorithm (based on TEND paper Algorithm 1 + FK graph from Phase 5A):

```

For each database (166 total):
1. Load FK graph (Phase 5A output)
2. For each table T:
    a. Classify relationship type from FK graph:
        - "leaf table" (T has no outgoing FKs): standalone collection
        - "bridge/junction table" (T connects two parent tables via FKs):
            flatten – embed as array in the primary parent, or keep as separate collection
        - "central table" (high in-degree centrality): primary collection, others
            reference it
3. Create MongoDB collection schema:
    - Collection name = table name (snake_case)
    - Fields = column names
    - ObjectId = primary key column
    - References = FK columns (store as ObjectId references initially – keep it simple
      for v1)
4. Load SQLite rows → insert as MongoDB documents
5. Save schema config to data/mongodb/{db_name}_schema.json

```

v1 simplification: Keep all tables as separate collections with reference-based FK relationships. This avoids complex embedding decisions and produces clean \$lookup-based MQL. Revisit embedding in Phase 19 (iteration).

Load into local MongoDB:

```

from pymongo import MongoClient

```

```

client = MongoClient('localhost', 27017)

for db_name in all_databases:
    db = client[db_name]
    for collection_name, docs in converted_data[db_name].items():
        db[collection_name].insert_many(docs)

```

Output: 166 MongoDB databases on local instance, data/mongodb/ schemas cached.

Verify:

```

from pymongo import MongoClient
client = MongoClient()
db = client['concert_singer']
print(db.list_collection_names()) # ['singer', 'concert', 'stadium', 'singer_in_concert']
print(db.singer.count_documents({})) # should match SQLite row count

```

Risk: SQLite data may have type inconsistencies (INT stored as TEXT, etc.). Write type coercion in converter. Test on 5 databases first.

PHASE 6 — PromptSchema via BM25S [■, Mac]

Why: SchemaRAG demonstrates that ambiguous column names (reg_dt, c_nm, amt) cause schema linking failures when the model has no semantic context. PromptSchema uses BM25S to auto-extract representative sample values, disambiguating column semantics for the LLM.

What it produces (per column/field):

```

{
  "singer.Age": {
    "sample_values": [23, 35, 41, 28],
    "inferred_type": "integer",
    "description": "Age of the singer in years"
  },
  "singer.Nationality": {
    "sample_values": ["France", "USA", "UK"],
    "inferred_type": "string",
    "description": "Country of origin"
  }
}

```

Implementation (src/prompt_schema.py):

```

import bm25s, sqlite3

class PromptSchemaBuilder:
    def build_sql(self, db_path, db_name) -> dict:
        conn = sqlite3.connect(f"{db_path}/{db_name}/{db_name}.sqlite")
        result = {}
        for table in self._get_tables(conn):
            for col in table['columns']:
                values = conn.execute(
                    f"SELECT DISTINCT [{col}] FROM [{table['name']}] "
                    f"WHERE [{col}] IS NOT NULL LIMIT 20"
                ).fetchall()
                values = [v[0] for v in values]
                if len(values) < 3:
                    continue # skip uninformative columns
                # BM25S: score values by representativeness
                top_values = self._bm25s_sample(values, top_k=3)
                result[f"{table['name']}.{col}"] = {"sample_values": top_values}
        return result

    def build_nosql(self, db_name, collection_name, sample_docs) -> dict:

```



```
# Same logic but over MongoDB document fields
...
```

Build for all 166 databases:

- SQL: cache to data/prompt_schema/sql/{db_name}.json
- NoSQL: cache to data/prompt_schema/nosql/{db_name}.json

Verify: prompt_schema['singer.Nationality'] returns real country values, not None.

CP1 Deliverable prerequisite: PromptSchema complete means data pipeline is fully operational.

PHASE 7A — SQL RAG Corpus Construction [■, Mac]

Why: The SAR (Schema-Augmented Retriever) is trained with contrastive learning — it needs to know which (Question, SQL) pairs are "structurally similar" (positive pairs) and which are not (negative pairs). We build this structural grouping from Spider's existing 7000 Q-SQL pairs.

Process:

1. Parse every SQL in Spider train with sqlglot
2. Assign a 6-dimensional "structural type" vector per query:

```
```python
```

```
structural_type = {
'num_joins': count_joins(ast), # 0, 1, 2, 3+
'has_group_by': bool,
'has_order_by': bool,
'has_having': bool,
'has_subquery': bool,
'num_tables': count_tables(ast)
}
```

```
```
```

3. Group by structural similarity → defines positive pairs for contrastive learning
4. Store as data/rag_corpus/spider_sql_rag.json:

```
```json
```

```
{"question": "...", "sql": "...", "db_name": "...", "structural_type": {...}}
...
```

**Note:** Spider's 7000 train examples are sufficient as base RAG corpus — SchemaRAG used Spider + GPT-4o extensions. Start with Spider alone; if SAR retrieval quality is poor in Phase 18, extend with DeepSeek-generated pairs.

**Verify:** At least 5 structural type categories populated; sqlglot parses all 7000 SQLs without error

---

## PHASE 7B — NoSQL RAG Corpus Construction [■, Mac + API]

**Why:** For Text-to-NoSQL, we need (Question, MQL, MongoDB Schema) triples. These don't exist pre-built. We generate them using the Spider questions + our MongoDB schemas + DeepSeek-V3 as translator.

**Strategy:** Use DeepSeek-V3 (not GPT-4o — ~10× cheaper at \$0.0003/1K tokens) to translate Spider's Q-SQL pairs to Q-MQL pairs, then verify by execution on local MongoDB.

**Process:**

```

prompt = f"""
You are a MongoDB expert. Convert this SQL query to a MongoDB aggregation pipeline
(PyMongo format).

Database collections available: {mongodb_schema}
SQL: {sql_query}
Question: {question}

Return ONLY the PyMongo command, e.g.:
db.collection.aggregate([...])
"""

Call DeepSeek-V3 API
Execute returned MQL on MongoDB
Compare result to SQLite execution result
Keep only verified matches

```

**Target:** ~4,000–5,000 verified Q-MQL pairs (expect ~60–65% verification pass rate on simple queries, lower on complex).

**Processing order:** Start with 1-table queries (easiest), then 2-table (with \$lookup), then 3+ table. Stop early if time-constrained.

**Cost estimate:** 7000 pairs × 800 tokens avg × \$0.0003/1K = ~\$1.68 — negligible.

**Store as:** data/rag\_corpus/spider\_nosql\_rag.json

**Verify:** Load 10 random entries, execute MQL on MongoDB, confirm non-empty results

## PHASE 8A — SQL SchemaLinker CoT Data Generation [■, Mac + API]

**Why:** SchemaRAG's SchemaLinker training starts with high-quality Chain-of-Thought reasoning data distilled from a teacher model (GPT-4o). The CoT teaches the student model HOW to reason about schema, not just what to output.

**Teacher prompt** (from SchemaRAG Section 3.2.2):

```

You are a schema linking expert. Given a question and a database schema,
identify the relevant tables and columns needed to answer the question.
Think step by step, reasoning through what the question requires.
Reference this ground-truth SQL to guide your reasoning: {ground_truth_sql}

Question: {question}
Database Schema with sample values:
{prompt_schema}

Output format:
<reasoning>Step-by-step analysis...</reasoning>
<linked_tables>table1, table2</linked_tables>
<linked_columns>table1.col1, table2.col2</linked_columns>

```

**Filtering rule** (Equation 4 from paper): Only keep CoT examples where every table and column mentioned in <linked\_tables> and <linked\_columns> EXACTLY matches entities in the ground-truth SQL. Discard the rest.

**Expected yield:** ~4,000–5,000 from 7,000 inputs (60–70% pass filtering).

**Cost:** Using DeepSeek-V3 — ~7000 × 1500 tokens × \$0.0003/1K = ~\$3.15

**Store as:** data/cot\_data/sql\_cot\_train.json

**Verify:** Random sample of 10 — check reasoning mentions correct table names; linked entities match SQL exactly

## PHASE 8B — NoSQL SchemaLinker CoT Data Generation [■, Mac + API]

Same process as 8A but for MongoDB schemas.

- Input: Q-MQL pairs from Phase 7B (~4000–5000)
- Teacher prompt: question + MongoDB PromptSchema + ground-truth MQL as hint
- Output: CoT reasoning + linked collections + linked fields
- Filtering: linked collections/fields must exactly match those in ground-truth MQL
- Expected yield: ~2,500–3,500 valid CoT examples

**Key difference in output format:**

```
<reasoning>The user asks about singers from France. I need the singer collection,
filtering on the Nationality field...</reasoning>
<linked_collections>singer</linked_collections>
<linked_fields>singer.Singer_ID, singer.Name, singer.Nationality</linked_fields>
```

Store as: data/cot\_data/nosql\_cot\_train.json

---

## ■■ CHECK: Before training phases, verify from SchemaRAG GitHub

**Critical shortcut:** SchemaRAG is open-source at <https://github.com/chelsea2002/SchemaRAG>. Before running Phases 8A/8B, **check if they released their CoT training data and RAG corpus**. If yes, download directly — saves ~\$7 in API costs and several days of generation time.

```
git clone https://github.com/chelsea2002/SchemaRAG external/SchemaRAG
ls external/SchemaRAG/data/ # check for training data
```

If their data covers Spider and is in compatible format, skip Phase 8A and use their data directly.

---

## PHASE 9A — SchemaLinker Stage 1: SQL CoT SFT [■, Colab T4]

**Objective:** Fine-tune Qwen-7B on SQL CoT data. After this, the model can produce chain-of-thought schema reasoning.

**Base model:** Qwen/Qwen2.5-7B (NOT Qwen2.5-Coder — general reasoning > code generation for schema linking, per SchemaRAG design)

**LoRA config** (fits in T4 16GB):

```
LoraConfig(
 r=16, lora_alpha=32,
 target_modules=["q_proj", "k_proj", "v_proj", "o_proj"],
 lora_dropout=0.05, bias="none", task_type="CAUSAL_LM"
)
```

**Training:**

- Epochs: 3
- Batch: 4 + gradient accumulation 4 = effective 16
- LR: 2e-4 (LoRA)
- Max seq length: 2048
- Loss: cross-entropy on assistant tokens only (not prompt)

**Input/output format:**

```
[SYSTEM] You are a schema linking expert for SQL databases.
[USER] Question: {question}
Database Schema:
{prompt_schema_output}
[ASSISTANT] <reasoning>...</reasoning>
```

```
<linked_tables>table1, table2</linked_tables>
<linked_columns>table1.col1, table2.col2</linked_columns>
```

**Evaluation** (Spider dev, 1034 examples):

- Table F1 (target from SchemaRAG Appendix B: >0.85 at k=1)
- Column Recall (target: >0.70 at k=1)

**Script:** Write scripts/train\_sl\_stage1\_sql.py on Mac → push → run on Colab T4

**Save:** Drive/codegen/checkpoints/sl\_sql\_stage1/

---

## PHASE 9B — SchemaLinker Stage 1: NoSQL CoT SFT [■, Colab T4]

Same as 9A but on NoSQL CoT data.

**Transfer learning trick:** Initialize from SQL Stage-1 checkpoint (Phase 9A). The reasoning patterns are similar — only the schema vocabulary differs (collections/fields instead of tables/columns). This should significantly reduce training time and improve convergence.

**Fine-tune from** Drive/codegen/checkpoints/sl\_sql\_stage1/ on NoSQL CoT data (~2500–3500 examples).

**Evaluation:** Collection Recall > 0.80, Field Recall > 0.65 on NoSQL dev set

**Save:** Drive/codegen/checkpoints/sl\_nosql\_stage1/

---

## PHASE 10A — SchemaLinker Stage 2: SQL Multi-Task Alignment [■, Colab T4]

**Objective:** Build an error dataset and train the model to detect and correct its own schema linking mistakes.

**Why this stage matters:** SchemaRAG ablation shows that removing SchemaLinker drops EX by ~3–5 points. Stage 2 MTL is what makes SchemaLinker robust — it explicitly teaches error recovery.

**Error dataset construction** (SchemaRAG Section 3.2.3):

1. Run Stage-1 model on ALL 7000 Spider train examples × 5 epochs
2. Collect every example where Stage-1 prediction ≠ ground truth → candidate error set
3. Manual curation: filter to cases where wrong schema → wrong SQL (not just formatting differences)
4. Target: ~500–800 curated error examples
5. Annotate with DeepSeek: (wrong prediction + error explanation + corrected reasoning)

**Multi-task training dataset** (3 tasks):

Task 1 – Error Detection (weight 3):

Input: question + schema + [WRONG PREDICTION from Stage-1]  
Output: "Error detected: {explanation of what is wrong}"

Task 2 – Corrective Analysis (weight 3):

Input: question + schema + [WRONG PREDICTION] + [ERROR EXPLANATION]  
Output: correct reasoning + correct linked entities

Task 3 – Answer Generation (weight 10):

Input: question + schema  
Output: correct reasoning + correct linked entities (standard generation)

**Sampling ratio 3:3:10** — Task 3 dominates because that is the primary inference task (Equation 7 from paper).

**Save:** Drive/codegen/checkpoints/sl\_sql\_stage2/

---

## PHASE 10B — SchemaLinker Stage 2: NoSQL MTL [■, Colab T4]

Same as 10A on NoSQL error data. Initialize from `sl_nosql_stage1`.

Error detection for NoSQL: wrong collection selection, wrong field, missing \$lookup relationship.

**Save:** `Drive/codegen/checkpoints/sl_nosql_stage2/`

---

## PHASE 11A — SchemaLinker Stage 3: SQL GRPO [■, Colab A100]

**Objective:** Apply Group Relative Policy Optimization to push schema linking accuracy to maximum via reward-based learning. This is the most computationally expensive step.

**Why GRPO:** Classic SFT optimizes token-level cross-entropy. GRPO directly optimizes the end metric (correct schema linking) — analogous to how RLHF improved chat models. SchemaRAG shows GRPO adds ~5–8 points in Table/Column Recall on top of Stage 2.

**GRPO details** (Equations 9–11 from SchemaRAG):

For each question + schema:

1. Generate  $G=8$  candidate schema linkings from current policy
2. Score each against ground truth using reward function  $R$ :  
$$R = rc \cdot |Ps \cap Ts| + rw \cdot |Ps - Ts| + rm \cdot |Ts - Ps| + wf \cdot F1$$
  
where:  $rc=+2$  (TP reward),  $rw=-0.5$  (FP mild penalty),  
 $rm=-3$  (FN STRONG penalty – missing tables is worst),  
 $wf=+0.5$  (F1 bonus)
3. Compute relative advantages within the group of  $G=8$  outputs
4. Update via GRPO objective (Equation 9 from paper)
5. KL penalty ( $\beta$ ) keeps policy close to reference model

**Why FN penalty is strongest:** Missing a required table makes correct SQL generation impossible. Including an extra table is tolerable — the decoder can ignore it.

**Hardware requirement:**  $G=8$  samples  $\times$  Qwen-7B  $\approx 7B \times 8$  forward passes simultaneously. With int4 quantization:  $7B \times 0.5$  bytes  $\times 8 = 28GB$  minimum. **Requires Colab A100 (40GB).**

**Self-consistency calibration** (replicate SchemaRAG Figure 10):

- After GRPO, test with  $k=1, 3, 5, 7$  self-consistency runs
- Performance plateaus around  $k=6-7$  (per paper)
- Use  **$k=5$**  for evaluation,  **$k=1$**  for demo (37s vs 4s latency)

**Target metrics** (from SchemaRAG Appendix B,  $k=5$ ):

- Table Recall  $> 0.96$
- Column Recall  $> 0.80$
- Column MCC  $> 0.85$

**Save:** `Drive/codegen/checkpoints/sl_sql_final/` (this is the production SchemaLinker for SQL)

---

## PHASE 11B — SchemaLinker Stage 3: NoSQL GRPO [■, Colab A100]

Same as 11A on NoSQL data. Reward function unchanged — FN (missing a collection/field) is equally harmful for MQL generation.

**Save:** `Drive/codegen/checkpoints/sl_nosql_final/`

---

## PHASE 12A — SAR Training for SQL [■, Colab T4]

**Objective:** Train the Schema-Augmented Retriever — the component that produces embeddings capturing SQL structural intent, not just surface text similarity. SchemaRAG ablation shows SAR is the MOST critical component (removing it drops EX by ~8–16 points across models).

**Architecture** (SchemaRAG Section 3.3):

#### Stage 1 — Schema-Aware Representation Learning:

Input: (question, table embeddings, column embeddings) via bge-large-en-v1.5 encoder

Step 1: Column-aware table embedding

$T^{\wedge}C_i = \text{Attention}(T_i, C_i, C_i)$  ← table attends to its own columns (Eq. 19)

Step 2: Question-schema fusion

$\blacksquare_i = \text{Attention}(E_q, T^{\wedge}C, T^{\wedge}C)$  ← question attends to all column-aware tables (Eq. 20)

Training loss:  $\text{MSE}(\blacksquare_i, \text{Encoder}(\text{ground\_truth\_SQL}))$  ← align schema-aware embed with SQL embed (Eq. 21)

#### Stage 2 — Contrastive Enhancement:

Input: [question\_embed; schema\_aware\_embed] → shape (2, batch, dim)

Causal mask: question can attend to schema, schema CANNOT attend back (Eq. 23)

This prevents reciprocal distortion while allowing schema-conditioning

3-layer, 8-head Transformer (optimal for Spider per SchemaRAG Fig. 14)

Output: Enhanced embedding  $E_{\text{final}}$

Positive pairs: Q-SQL pairs with same structural type vector (from Phase 7A)

Negative pairs: all other samples in batch (in-batch negatives)

Loss =  $L_{\text{contrastive}} + 0.5 \times L_{\text{similarity}}$  (Eq. 30)

Hyperparams: LR=1e-4, contrastive\_temperature=0.05 (from SchemaRAG Fig. 13)

**Evaluation target** (from SchemaRAG Section 4.6.2):

- Silhouette Score > 0.78 on test set (paper reports 0.42 before contrastive, 0.78 after)

**Script:** scripts/train\_sar\_sql.py

**Save:** Drive/codegen/checkpoints/sar\_sql/

---

## PHASE 12B — SAR Training for NoSQL [■, Colab T4]

Same architecture, smaller dataset (~4000 Q-MQL pairs).

**Architecture adjustment:** Use **2 layers, 2 heads** (leaner model performs better on smaller/harder datasets, per SchemaRAG Figure 14 — BIRD dataset behavior mirrors small NoSQL dataset).

**Positive pairs:** Q-MQL pairs with similar MQL operator structure (similar \$group, \$lookup patterns).

**Training loss:** MSE targets MongoDB query embedding (instead of SQL embedding).

**Save:** Drive/codegen/checkpoints/sar\_nosql/

---

## PHASE 13 — ChromaDB Index Building [■, Mac]

**Objective:** Build the vector stores that SAR will query at inference time to retrieve few-shot examples.

**Process:**

```
SQL index
for each (question, sql, db_name) in sql_rag_corpus:
 embedding = sar_sql.encode(question, db_schema=load_schema(db_name))
```

```

chroma_sql.add(embedding, metadata={'question': q, 'sql': sql, 'db_name': db_name})

NoSQL index
for each (question, mql, db_name) in nosql_rag_corpus:
 embedding = sar_nosql.encode(question, db_schema=load_nosql_schema(db_name))
 chroma_nosql.add(embedding, metadata={'question': q, 'mql': mql, 'db_name': db_name})

```

**Retrieval test** (critical verification):

```

Query: "How many singers are there?"
Expected: top-3 are other COUNT(*) queries, NOT complex JOIN queries
top3 = chroma_sql.query(
 sar_sql.encode("How many singers are there?", schema=concert_singer_schema),
 n_results=3
)
for ex in top3:
 print(ex['sql']) # should all be simple COUNT queries

```

**Save:** indexes/chroma\_sql/ and indexes/chroma\_nosql/

## PHASE 14A — SQL Generator Fine-tuning [■, Colab A100]

**Objective:** Fine-tune Qwen2.5-Coder-7B-Instruct to generate SQL given the full SchemaRAG prompt.

**Base model:** Qwen/Qwen2.5-Coder-7B-Instruct — chosen for strong code/SQL benchmark performance.

**Prompt format** (from SchemaRAG Figure 7):

```

You are a SQL expert. Generate a single SQL query.

Database: {db_name}
Schema (with sample values):
{prompt_schema_output}

Relevant schema elements:
{schema_linker_output}

Similar examples:
Q: {example1_question} | SQL: {example1_sql}
Q: {example2_question} | SQL: {example2_sql}
Q: {example3_question} | SQL: {example3_sql}

Question: {question}
SQL:

```

**Build training dataset:** For each of 7000 Spider train examples:

- Run Phase 6 PromptSchema (pre-computed, just load)
- Run Phase 11A SchemaLinker (pre-computed, just load)
- Run Phase 12A SAR retrieval (get top-3 examples)
- Construct full prompt → target = ground-truth SQL

**LoRA fine-tuning** (A100 required for batch size):

- LoRA r=16, alpha=32, target: q/k/v/o projections
- Epochs: 3, LR: 1e-4, max\_seq: 2048
- **Note:** Training with pre-computed SchemaLinker + SAR outputs makes this straightforward fine-tuning, not full pipeline training.

**Save:** Drive/codegen/checkpoints/generator\_sql/

**Target:** >82% EX on Spider dev without POSG (POSG adds ~1–2 points)

## PHASE 14B — NoSQL Generator Fine-tuning [■, Colab A100]

**Initialize from:** SQL generator checkpoint (Phase 14A) — SQL knowledge transfers to MQL.

**Output format:** PyMongo aggregation pipeline:

```
db.singer.aggregate([
 {"$match": {"Nationality": "France"}},
 {"$count": "total"}
])
```

**Training data:** Q-MQL pairs from Phase 7B with full SchemaLinker + SAR NoSQL context prepended to prompt.

**Save:** Drive/codegen/checkpoints/generator\_nosql/

**Target:** >60% EX on NoSQL dev set (matching TEND SMART framework)

---

## PHASE 15A — POSG for SQL [■, Mac]

**Objective:** Implement Pareto-Optimal SQL Generator. Given N=5 SQL candidates, select the best one.

**Algorithm** (SchemaRAG Equations 31–35):

```
def select_best_sql(candidates, schema_linked, few_shot_examples, db_path):
 # Step 1: Filter by executability (hard filter)
 executable = []
 for sql in candidates:
 try:
 conn = sqlite3.connect(db_path)
 conn.execute(sql).fetchall()
 executable.append(sql)
 except:
 pass # discard non-executable

 if not executable:
 return candidates[0] # fallback: return first even if non-executable

 # Step 2: Score on 2 dimensions
 scores = []
 for sql in executable:
 # Dimension 1: Schema linking conformity – Jaccard similarity (Eq. 33)
 sql_tables_cols = extract_schema_used(sql) # via sqlglot
 ssl = len(sql_tables_cols & schema_linked) / len(sql_tables_cols | schema_linked)

 # Dimension 2: Example consistency – avg normalized AST edit distance (Eq. 34)
 sql_ast = sqlglot.parse_one(sql)
 sec = mean([1 - ast_distance(sql_ast, parse_one(ex['sql']))
 for ex in few_shot_examples])

 scores.append({'sql': sql, 'ssl': ssl, 'sec': sec})

 # Step 3: Pareto front – find non-dominated candidates (Eq. 35)
 pareto = find_pareto_front(scores, dims=['ssl', 'sec'])

 # Step 4: From Pareto set, pick highest schema conformity (ssl)
 return max(pareto, key=lambda x: x['ssl'])['sql']
```

**AST distance:** Use sqlglot's AST representation with tree edit distance via zss library.

---

## PHASE 15B — POSG for NoSQL [■, Mac]



**Same algorithm, different execution environment.**

**Key challenge:** MQL has no standardized AST parser unlike SQL. Solution:

```
def mql_structural_distance(mql1: dict, mql2: dict) -> float:
 """JSON tree edit distance between two MQL pipelines."""
 import zss
 tree1 = dict_to_tree(mql1) # convert MQL dict to tree structure
 tree2 = dict_to_tree(mql2)
 return zss.simple_distance(tree1, tree2)

def is_executable_mql(mql_str: str, mongo_db) -> bool:
 """Try executing MQL on MongoDB."""
 try:
 exec(mql_str, {'db': mongo_db}) # eval PyMongo command
 return True
 except:
 return False
```

**Additional dependency:** `pip install zss --break-system-packages`

---

## PHASE 16 — End-to-End Pipeline Assembly [■, Mac]

**Objective:** Wire all components into complete inference pipelines.

**SQL Pipeline** (src/pipeline\_sql.py):

```
class TextToSQLPipeline:
 def generate(self, question: str, db_name: str, k_sc: int = 1) -> str:
 # 1. Load PromptSchema (pre-cached)
 prompt_schema = self.prompt_schema.load(db_name)

 # 2. SchemaLinker with self-consistency
 schema_linked = self.schema_linker.link(
 question, prompt_schema, k=k_sc # k=1 for demo, k=5 for eval
)

 # 3. SAR: retrieve top-3 examples
 examples = self.sar.retrieve(question, db_name, schema_linked, k=3)

 # 4. Generator: produce N=5 SQL candidates
 candidates = self.generator.generate(
 question, prompt_schema, schema_linked, examples, n=5
)

 # 5. POSG: select best candidate
 return self.posg.select(candidates, schema_linked, examples, db_name)
```

**NoSQL Pipeline** (src/pipeline\_nosql.py): Same structure, NoSQL component variants.

**Integration test:**

```
SQL test
pipeline_sql = TextToSQLPipeline(config)
sql = pipeline_sql.generate("How many singers are from France?", "concert_singer")
print(sql) # SELECT COUNT(*) FROM singer WHERE Nationality = 'France'

NoSQL test
pipeline_nosql = TextToNoSQLPipeline(config)
mql = pipeline_nosql.generate("How many singers are from France?", "concert_singer")
print(mql) # db.singer.count_documents({"Nationality": "France"})
```

---

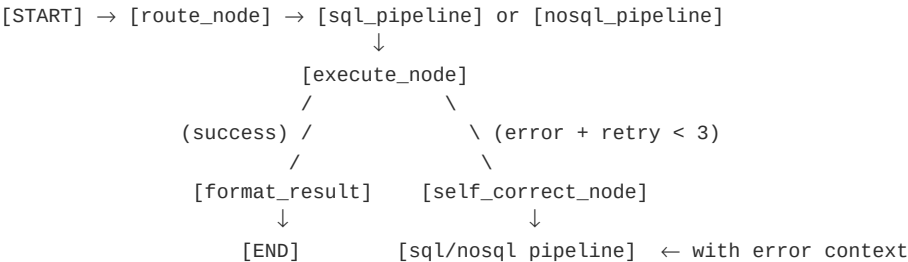
PHASE 17 — LangGraph Router + Self-Correction [■, Mac]

**Objective:** Build the orchestration layer. Route queries to correct pipeline; retry failed queries.

**LangGraph state:**

```
class CodeGenState(TypedDict):
 question: str
 db_type: str # "sql" or "nosql" — set at session start
 db_name: str # which database to query
 generated_query: str
 execution_result: Optional[Any]
 error_message: Optional[str]
 retry_count: int # max 3
 final_answer: str
```

**Graph structure:**



**Self-correction prompt** (added to generator input on retry):

```
Previous query failed with error: {error_message}
Failed query: {failed_query}
Please generate a corrected query that avoids this error.
```

**Verify:** Test 4 scenarios manually:

- 1. SQL query succeeds first try
- 2. NoSQL query succeeds first try
- 3. SQL query fails → corrects on retry
- 4. SQL query fails 3 times → graceful error message

PHASE 18 — Evaluation [■, Mac + Colab]

**18A — Text-to-SQL Evaluation:**

Metric	Target	Notes
Spider EX	>85%	SchemaRAG + XiYanSQL-14B achieves 93.3%; we use Qwen2.5-Coder-7B
Spider EM	>65%	Exact match is harder

**Ablation study** (replicate SchemaRAG Table 5):

- 1. Full pipeline (baseline)
- 2. w/o SchemaLinker
- 3. w/o SAR
- 4. w/o POSG

Expected finding from paper: removing SAR causes the largest drop. Use this to justify your architectural choices in the capstone writeup.

**Self-consistency analysis** (replicate Figure 10): Run k=1,3,5,7 → plot Table Recall and Column Recall curves.

### 18B — Text-to-NoSQL Evaluation:

Metric	Target	Notes
MongoDB EX	>60%	TEND SMART benchmark; direct generation baseline
vs direct generation	>+10%	Justify SchemaLinker + SAR overhead

### 18C — CP1 Baseline (codegen-350M):

```
Salesforce codegen-350M: simple fine-tuning baseline
pip install transformers
Fine-tune on Spider train (just question → SQL, no SchemaRAG components)
Report: EX on Spider dev – expected ~45-55%
This establishes the "without schema-awareness" lower bound
```

### 18D — Latency Analysis (replicate SchemaRAG Table 4):

Component	Avg latency	Cumulative
PromptSchema (pre-cached)	~0.01s	~0.01s
SchemaLinker k=1	~4s	~4s
SchemaLinker k=5	~20s	~20s
+ SAR retrieval	~0.5s	~20.5s
+ Generator (5 candidates)	~15s	~35.5s
+ POSG	~0.5s	~36s

Target: p95 latency < 40s with k=5. Demo uses k=1 (target: < 8s).

**Document:** evaluation/results\_v1.md with tables matching paper format.

---

## PHASE 19 — Error Analysis + Iteration [■, Mac + Colab]

**Objective:** Analyze failures, identify root causes, iterate.

**Error taxonomy** (from SchemaRAG Figure 9 — 300 sample analysis):

- Schema Errors (34.5%): wrong table, wrong column
- Data Analysis Errors (40.8%): wrong aggregation, incorrect calculation
- Other Errors (24.7%): join errors, condition filter errors

**Decision tree for iteration:**

IF Schema Errors > 40%:

- Retrain SchemaLinker with more GRPO iterations
- Check if PromptSchema is providing misleading samples

IF Data Analysis Errors > 45%:

- Add more complex Spider examples to RAG corpus
- Improve generator prompt (add explicit reasoning steps)

IF NoSQL EX < 55%:

- Check MongoDB collection conversion quality (Phase 5B)
- Verify Q-MQL pairs are correct (Phase 7B)
- Consider embedding-based MongoDB schema (not just reference-based)

IF Join Errors > 10%:

- Verify FK graph is capturing all relationships
- Add FK path explicitly to SchemaLinker prompt

**Document:** evaluation/iteration\_log.md:

- ■ What worked
  - ■ What failed
  - ■ New insights from failures
- 

## PHASE 20 — Demo + CP4 Scope [■, Mac]

### 20A — Streamlit Demo:

```
st_app.py
import streamlit as st

st.title("CodeGen: Natural Language to Database Query")

Session config
db_type = st.radio("Target database", ["PostgreSQL (SQL)", "MongoDB (NoSQL)"])
db_name = st.selectbox("Database", spider_db_names)
question = st.text_area("Your question", "How many singers are from France?")

if st.button("Generate Query"):
 with st.spinner("Generating..."):
 if db_type == "PostgreSQL (SQL)":
 query = pipeline_sql.generate(question, db_name, k_sc=1)
 else:
 query = pipeline_nosql.generate(question, db_name, k_sc=1)

 st.code(query)

Explainability sidebar
with st.expander("How it worked"):
 st.write("Schema linked:", pipeline.last_schema_linked)
 st.write("Retrieved examples:", pipeline.last_examples)
```

### 20B — FastAPI Backend (for CP4):

```
@app.post("/api/query")
async def generate_query(request: QueryRequest):
 # {question, db_type, db_name}
 pipeline = sql_pipeline if request.db_type == "sql" else nosql_pipeline
 query = pipeline.generate(request.question, request.db_name)
 result = execute(query, request.db_name, request.db_type)
 return {"query": query, "result": result}
```

### 20C — SQL-to-NoSQL Migration Utility (CP4 deliverable):

- Input: SQL query string
  - Process: detect SQL structure → run through NoSQL generator with SQL as context hint
  - Output: equivalent MongoDB MQL + explanation of collection mapping
  - Reuses Phase 5B MongoDB schemas + Phase 14B NoSQL generator
  - **Defer full implementation to CP4** — design only now
- 
- 

## SECTION 5 — RISKS AND MITIGATIONS

#	Risk	Likelihood	Impact	Mitigation
1	GRPO Stage 3 OOM on Colab free (T4 16GB)	High	High	Use Colab Pro A100 (40GB); reduce G from 8 to 4 as emergency fallback
2	NoSQL MongoDB conversion errors (type mismatches, null handling)	High	Medium	Type coercion layer in converter; test on 10 databases first; use VARCHAR fallback
3	Q-MQL verification pass rate < 40% (too few NoSQL training pairs)	Medium	High	Focus on 1–2 table queries first (higher pass rate); manually fix 200 complex cases
4	SchemaRAG CoT filtering rejects > 50% of inputs	Medium	Medium	Lower threshold: accept partial entity matches (all required entities present, extra OK); use DeepSeek with higher temperature for diversity
5	Self-consistency k=5 too slow for demo (37s)	High	Low	Demo uses k=1 (<8s); evaluation uses k=5 (37s); document tradeoff explicitly
6	MQL AST distance (zss) imprecise for POSG	Medium	Low	Acceptable for v1 — POSG is least critical component (SchemaRAG ablation: removing POSG causes smallest drop); revisit in Phase 19
7	SQLite→PostgreSQL dialect differences break production queries	Low	Medium	Use <code>sqlglot.transpile(sql, read='sqlite', write='postgres')</code> as thin wrapper; test on 20 complex queries
8	SAR overfits on small NoSQL corpus (~4000 pairs)	Medium	Medium	Leaner architecture (2L-2H); aggressive dropout (0.1); early stopping
9	GPT-4o/DeepSeek API rate limits during batch CoT generation	Low	Low	Use exponential backoff; process in batches of 100; cache immediately
10	SchemaRAG GitHub doesn't release training data	Medium	Medium	Run Phase 8A/8B with DeepSeek (~\$3 total); acceptable fallback
11	MongoDB not available on Colab for NoSQL POSG executability check	Medium	Low	Use mongomock Python library for Colab testing; real MongoDB only for Mac dev and demo

#	Risk	Likelihood	Impact	Mitigation
12	Qwen2.5-Coder-7B fine-tuning diverges (loss spikes)	Low	Medium	Start with smaller LR (5e-5); use warmup 100 steps; monitor validation loss every 500 steps

## SECTION 6 — CHECKPOINT DELIVERABLES MAPPING

### CP1 Deliverables

Item	Phase	Status
Spider dataset downloaded and validated	4	Pending
FK graph builder functional (all 166 DBs)	5A	Pending
MongoDB collection generator (all 166 DBs)	5B	Pending
PromptSchema built (SQL + NoSQL)	6	Pending
Evaluation framework (EX metric script)	18	Pending
Baseline: codegen-350M EX on Spider dev	18C	Pending
NoSQL RAG corpus construction started	7B	Pending

**CP1 Target EX:** codegen-350M baseline ~45–55% (the "without schema-awareness" floor)

### CP2 Deliverables

Item	Phase	Status
SchemaLinker trained (SQL, all 3 stages)	9A–11A	Pending
SchemaLinker trained (NoSQL, all 3 stages)	9B–11B	Pending
SAR trained (SQL)	12A	Pending
SAR trained (NoSQL)	12B	Pending
Qwen2.5-Coder-7B fine-tuned (SQL)	14A	Pending
Qwen2.5-Coder-7B fine-tuned (NoSQL)	14B	Pending
POSG implemented (SQL + NoSQL)	15A–15B	Pending
Text-to-SQL EX > 82% on Spider dev	18A	Pending
Text-to-NoSQL EX > 55% on MongoDB dev	18B	Pending

### CP3 Deliverables

Item	Phase	Status
ChromaDB indices built (SQL + NoSQL)	13	Pending

Item	Phase	Status
End-to-end pipelines assembled	16	Pending
LangGraph router + self-correction	17	Pending
Full ablation study documented	18	Pending
Text-to-SQL EX > 85% on Spider dev	18A	Pending
Text-to-NoSQL EX > 60% on MongoDB dev	18B	Pending
Error analysis + first iteration complete	19	Pending

## CP4 Deliverables

Item	Phase	Status
Streamlit demo working	20A	Pending
FastAPI backend	20B	Pending
SQL-to-NoSQL migration utility	20C	Pending
Final evaluation report (paper-format)	19	Pending

## SECTION 7 — DESIGN DECISIONS LOG

Decision	Choice	Rationale
Routing strategy	Session-based (Option A)	Simpler than conversational routing; no ambiguity; sufficient for capstone scope
Text-to-NoSQL approach	Direct generation (SMART)	TEND paper: direct 65% EX vs cascade 44% EX — a 21-point gap too large to ignore
NoSQL dataset source	Spider SQLite → MongoDB conversion	Reuses existing verified 7000 Q-SQL pairs; avoids building dataset from scratch
SchemaLinker base model	Qwen/Qwen2.5-7B (general)	CoT reasoning requires general intelligence; Qwen2.5-Coder is better at code output (saved for generator)
Generator base model	Qwen/Qwen2.5-Coder-7B-Instruct	Best open-source SQL benchmark performance (per SchemaRAG paper Table 2: 93.3% with XiYanSQL-14B, 80.4% with Qwen-7B)
SQL execution backend	SQLite (Spider's native format)	Avoids dialect conversion during training; sqlglot handles PostgreSQL conversion for production
GRPO reward weights	FN: -3, TP: +2, FP: -0.5, F1: +0.5	Exact values from SchemaRAG Eq. 11; missing a table is unrecoverable
SAR architecture (SQL)	3 layers, 8 heads	SchemaRAG Figure 14: deeper model better for Spider-complexity
SAR architecture (NoSQL)	2 layers, 2 heads	Figure 14: leaner model better for harder/smaller datasets (mirrors BIRD behavior)

Decision	Choice	Rationale
Self-consistency k	k=5 eval, k=1 demo	SchemaRAG Figure 10: k=5–7 optimal; k=1 for demo latency (<8s)
NoSQL MQL format	PyMongo aggregation pipeline	Executable natively; structured JSON enables structural distance comparison
POSG fallback	Return first candidate if none executable	Prevents total pipeline failure; acceptable for demo
NoSQL SchemaLinker init	Transfer from SQL Stage-1	Reasoning patterns identical; only vocabulary changes; accelerates convergence
MQL AST distance	JSON structural edit distance (zss)	No standard MQL AST parser exists; zss provides approximate structural similarity
Teacher model for CoT	DeepSeek-V3	~10× cheaper than GPT-4o; comparable quality for structured CoT tasks
MongoDB schema strategy (v1)	Reference-based (separate collections)	Simpler for v1; embedding strategy deferred to Phase 19 iteration if EX is low

## SECTION 8 — KEY NUMBERS TO REMEMBER

From SchemaRAG paper (your performance targets and baselines):

Configuration	Spider EX	BIRD EX
SchemaRAG + GPT-4o	85.6%	68.9%
SchemaRAG + DeepSeek-V3	89.6%	68.4%
SchemaRAG + Qwen-7B	<b>80.4%</b>	<b>54.5%</b>
SchemaRAG + XiYanSQL-14B	93.3%	65.3%
Best baseline (TA-SQL + GPT-4o)	85.0%	52.4%
w/o SAR (Qwen-7B)	72.5%	46.7%
w/o SchemaLinker (Qwen-7B)	78.1%	55.2%

**Your realistic target:** 80–82% Spider EX with Qwen-7B SchemaLinker + Qwen2.5-Coder-7B generator (between Qwen-7B baseline and DeepSeek-V3 in the paper).

From TEND paper (NoSQL targets):

Configuration	MongoDB EX
SMART (direct Text-to-NoSQL)	<b>65.08%</b>
LLM SQL→NoSQL (with correct SQL)	44.76%
Grammar-based SQL→NoSQL	10.81%

**Your realistic target:** 55–65% MongoDB EX (your dataset is smaller, but architecture mirrors SMART).



## SECTION 8B — REVIEW: LOOPHOLES IDENTIFIED AND ADDRESSED

These were found during critical review. They are addressed within the phases above, but flagged here explicitly.

#	Loophole	Found Where	Resolution in Plan
L1	SchemaRAG uses SQLite for ALL execution (not PostgreSQL). Spider DBs are SQLite. Training on PostgreSQL would require converting 166 DBs.	Phase 14A	Use SQLite throughout training + evaluation. Add <code>sqlglot.transpile(sql, read='sqlite', write='postgres')</code> as thin production wrapper only.
L2	GRPO Stage 3 needs G=8 forward passes of Qwen-7B simultaneously ≈ 28GB GPU RAM minimum. Colab free T4 (16GB) cannot run this.	Phase 11	<b>Colab A100 (Pro) required</b> — explicitly flagged. Emergency fallback: G=4 (halves GPU requirement).
L3	Text-to-NoSQL training dataset doesn't exist publicly. TEND paper built theirs privately.	Phase 7B	Generate via DeepSeek-V3 API (~\$1.68) + MongoDB execution verification. Accept 60-65% pass rate.
L4	SchemaLinker Stage 2 MTL requires manual curation of error examples.	Phase 10A	Flagged explicitly. Budget 2-3 hours for manual review of ~200 error cases. Cannot be skipped — paper says automatic filtering misses cases where wrong schema → correct SQL.
L5	Some Spider databases have NO explicit foreign keys defined (natural joins only). FK graph has no edges.	Phase 5A	Implement fallback: if no FK edges, use co-occurrence of table names across training SQLs as proxy FK signal.
L6	MongoDB is unavailable on Colab during evaluation (POSG Phase 15B needs live MongoDB for executability check).	Phase 15B	Use mongomock library for Colab evaluation. Real MongoDB only for Mac dev + demo. Note: <code>pip install mongomock</code>
L7	Spider official evaluation script is at <code>taoyds/spider/evaluation.py</code> . Do NOT implement custom EX metric — use official to ensure comparability.	Phase 18A	Use official Spider eval script. For NoSQL, adapt execution comparison: run both SQLite query and MQL, compare result sets.
L8	Training phases 9A→11A (SQL) and 9B→11B (NoSQL) look "parallel" but Colab is single-instance sequential. Plan implies parallelism that is physically impossible.	Phases 9-12	Execute ALL SQL training first (9A→10A→11A→12A), then ALL NoSQL training (9B→10B→11B→12B). NoSQL Stage 1 starts from SQL Stage 1 checkpoint anyway (transfer learning), so this ordering is correct AND advantageous.

#	Loophole	Found Where	Resolution in Plan
L9	CoT generation quality depends heavily on the teacher model. DeepSeek-V3 may produce lower-quality CoT than GPT-4o, resulting in worse SchemaLinker.	Phase 8A/8B	DeepSeek-V3 is GPT-4o class on structured reasoning. SchemaRAG's entity-level filtering step catches bad CoT regardless. If CoT yield < 50%, switch to GPT-4o-mini for a subset.
L10	Qwen2.5-Coder-7B generator fine-tuning uses pre-computed SchemaLinker + SAR outputs as training context. If SchemaLinker output format changes after GRPO, training data is stale.	Phase 14A	Generate final training prompts AFTER Phase 11A (GRPO) is complete, using the production SchemaLinker. Do NOT pre-generate in Phase 8.

## SECTION 9 — IMMEDIATE NEXT STEPS

Execute in this order after this plan is approved:

1. **Phase 3D:** Create `src/device.py` (10 min)
2. **Phase 3E:** Create all directories, `configs/config.yaml`, `.env`, `.gitignore`, commit (30 min)
3. **Phase 4:** Download Spider dataset (1–2 hours, mostly download time)
4. **Phase 5A:** Build FK graphs for all 166 databases (2–3 hours coding)
5. **Phase 5B:** Install MongoDB, convert 166 databases (1 day)
6. **Phase 6:** Build PromptSchema for all databases (3–4 hours)
7. **Check SchemaRAG GitHub** for released training data before Phase 8A

After data pipeline complete (CP1 boundary), shift to training phases on Colab.